The non-dependent recursor for the flattened total space and its definitional computation rule

The Univalent Foundations Program

hott-book:hits.tex:local-snapshot-20261011; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author's 'thm:flattening-rectnd' (source lines 1929-1936, Lemma 6.12.5); the proof environment starts on the next line, so the pairing is by adjacency.

Editorial note. The retained context is the author's own paragraph explaining why the dependent recursion principle is not enough and why only the non-dependent computation rule is needed, which is what makes the lemma's role visible.

Editorial note. The retained counter values (chapter 6, section 12, theorem offset 4, equation offset 5) make the retained lemma print as Lemma 6.12.5 and the numbered display inside its proof print as equation (6.12.6), matching the original book numbering.

Editorial note. The four external references are navigation pointers to results of other chapters and of the same section that the retained proof invokes by label; the counted proof is retained in full.

Editorial scope. One original result of the HoTT Book, counted once: the non-dependent recursor for the flattened total space with its computation rule, Lemma 6.12.5.

Allowed standard prerequisites (editorial).

- The flattening setup. P : W -> type is defined recursively along the path constructor of the coequalizer W, and W-tilde has constructors c-tilde(a,x) and p-tilde(b,y).. The lemma is stated for data c : prd a:A C a -> Q and p : prd b:B prd y:C(f b) (c(f b,y) = c(g b, D b y)) over that setup.
- Induction on W and dependent paths into a constant family. A map k out of Sigma w:W P(w) may be defined by induction on w, and for a constant family the coherence condition may be phrased with thm:dpath-arrow.. The definition at source lines 1938-1939 uses induction on W and the coherence obligation at line 1940 is converted by thm:dpath-arrow.
- Transport in a constant family and the canonical transport along the recursive definition of P. trans along a path in a constant family is trivial, and trans (p b) y = D b y by thm:transport-is-given.. They are the first and third steps of the displayed composite at source lines 1949-1952.

Source reference. Original lemma 6.12.4: sources/hott-hits/pinned-original-source.tex, source lines 1870–1870. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 2.9.6: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1764–1764. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 2.3.5: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1902–1902. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 6.12.1: sources/hott-hits/pinned-original-source.tex, source lines 1772–1772. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

original lemma 6.12.4 *almost* gives $\sum_{(w:W)} P(w)$ the same induction principle as $\widetilde{W}$. The missing bit is the equality $\mathsf{apd}_k(\widetilde{\mathsf{p}}'(b,y)) = p(b,y)$. In order to prove this, we would need to analyze the proof of original lemma 6.12.4, which of course is the definition of k.

It should be possible to do this, but it turns out that we only need the computation rule for the non-dependent recursion principle. Thus, we now give a somewhat simpler direct construction of the recursor, and a proof of its computation rule.

Lemma 6.12.6. *Suppose Q is a type and that we have*

- $c : \prod_{(a:A)} C(a) \to Q$ *and*
- $p : \prod_{(b:B)} \prod_{(y:C(f(b)))} \Big(c(f(b),y) =_Q c(g(b), D(b)(y))\Big)$.

Then there exists $k : \big(\sum_{(w:W)} P(w)\big) \to Q$ such that $k(\widetilde{\mathsf{c}}'(a,x)) \equiv c(a,x)$.

Proof. As in original lemma 6.12.4, we define $k(w,x)$ by induction on $w : W$. When $w \equiv \mathsf{c}(a)$, we define $k(\mathsf{c}(a),x) :\equiv c(a,x)$. Now by original lemma 2.9.6, it suffices to consider, for $b : B$ and $y : C(f(b))$, the composite path

$$\mathsf{transport}^{x\mapsto Q}(\mathsf{p}(b), c(f(b),y)) = c(g(b), \mathsf{transport}^{P}(\mathsf{p}(b),y)) \tag{6.12.7}$$

defined as the composition

$$\begin{aligned}
\mathsf{transport}^{x\mapsto Q}(\mathsf{p}(b), c(f(b),y)) &= c(f(b),y) && \text{(by original lemma 2.3.5)}\\
&= c(g(b), D(b)(y)) && \text{(by } p(b,y)\text{)}\\
&= c(g(b), \mathsf{transport}^{P}(\mathsf{p}(b),y)). && \text{(by original lemma 6.12.1)}
\end{aligned}$$

The computation rule $k(\widetilde{\mathsf{c}}'(a,x)) \equiv c(a,x)$ follows by definition, as before. □